FastAPI Sample Guide

A small technical document created for testing a RAG document-ingestion pipeline.

1. What is FastAPI?

FastAPI is a modern Python web framework used to build APIs. It supports type hints, automatic request validation, and automatic interactive API documentation.

2. Creating a FastAPI Application

A basic FastAPI application creates an application object and defines routes using decorators.

3. Example GET API

The following example creates a simple GET endpoint.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
async def root():
    return {"message": "Hello FastAPI"}
```

4. Example POST API

POST endpoints can receive structured request data. Pydantic models are commonly used to validate request bodies.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
async def create_user(user: User):
    return user
```

5. Path and Query Parameters

FastAPI supports path parameters and query parameters. Type annotations help FastAPI validate incoming values.

```
@app.get("/users/{user_id}")
async def get_user(user_id: int, active: bool = True):
    return {
        "user_id": user_id,
        "active": active
    }
```

6. Automatic API Documentation

FastAPI automatically generates interactive documentation. Swagger UI is normally available at /docs and ReDoc is normally available at /redoc.

7. File Uploads

FastAPI can receive uploaded files using UploadFile and File. Uploaded files can then be saved or processed by the application.

```
from fastapi import FastAPI, UploadFile, File

app = FastAPI()

@app.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    return {
        "filename": file.filename,
        "content_type": file.content_type
    }
```

8. FastAPI in a RAG Application

In a Retrieval-Augmented Generation application, FastAPI can expose APIs for document uploads and chat. A document upload endpoint can receive a PDF, extract its text, split the text into chunks, create embeddings, and store those embeddings in a vector database such as Qdrant. A chat endpoint can embed a user's question, retrieve relevant chunks, and send the retrieved context to an LLM.

9. Example RAG Flow

A typical document question-answering flow is: Upload document → Extract text → Split into chunks → Create embeddings → Store in Qdrant → Embed user question → Search Qdrant → Retrieve relevant chunks → Send context to Gemini → Return answer.